Agent 的实现范式对比

- 来源: https://class.imooc.com/lesson/5290#mid=105154
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: Agent的核心架构剖析
- 课节: 图文 1-5 Agent 的实现范式对比

1-5 Agent 的实现范式对比

Agent 的实现范式对比

> [!TIP] 阅读指南 学完了 Agent 的核心概念、架构、能力、记忆和推理模式，现在面临一个实际问题：如何实现一个 Agent？ 你会发现，同样是实现一个客服 Agent，有人用 100 行代码搞定，有人写了 5000 行还没跑通；有人直接调 API，有人引入了三个框架。这些差异的背后，是不同的实现范式选择。 本节我们聊聊 Agent 实现的几种主要方式，以及如何根据你的场景做出正确选择。

架构选择：单体 vs 模块化

单体 Agent：一个模型搞定一切

核心思想：用一个 LLM 承担所有角色——感知、理解、规划、执行，所有决策都在同一个 Prompt 的指导下完成，没有功能分工。

想象一个万能助手，你问什么它都要自己处理：查天气、订票、写代码、解答问题。...所有能力都塞在一个"脑子"里。

典型实现：

典型实现

```
# 单体Agent：一个LLM处理所有任务
def single_agent(user_input, chat_history):
    system_prompt = """
    你是一个全能助手，可以：
    1. 回答各类问题
    2. 帮助用户订票
    3. 查询天气信息
    4. 编写代码
    5. 分析数据

    请根据用户输入，判断需求并给出回复。
    """

    messages = [{"role": "system", "content": system_prompt}]
    messages.extend(chat_history)  # 可以有多轮对话
    messages.append({"role": "user", "content": user_input})

    # 关键：所有任务都由同一个LLM处理，没有功能分工
    response = llm.chat(messages)
    return response
```

优势在于简单直接、代码量少，部署方便、依赖少，架构简单、容易理解。

优势

劣势在于 LLM 负担重，容易"人格分裂"（一会儿是客服，一会儿是技术专家）；难以优化（所有逻辑混在一个 Prompt 里）；扩展性差（新增能力要重写整个 Prompt）；Prompt 容易臃肿（十几种能力写在一起）。

适用场景：任务单一明确（如情感分析 Agent）、预算有限追求低成本、快速验证原型。

模块化 Agent：专业分工，各司其职

核心思想：将 Agent 拆分为多个模块，每个模块专注一件事。

典型架构：

典型架构

优势在于每个模块可独立优化（用不同的模型、不同的 Prompt），扩展性强（新增能力只需加一个模块），可维护性好（问题出在哪个模块一目了然），成本可控（简单任务用小模型，复杂任务才上大模型）。

劣势在于复杂度高、开发成本大，多次 LLM 调用导致响应变慢，模块间协调需要额外设计。

适用场景：任务复杂多样（如综合客服系统）、对准确性要求高、长期维护的生产系统。

混合模式：务实的选择

实际项目中，很少纯单体或纯模块化，更多是混合——核心对话用单体（保持流畅），复杂任务拆模块（保证质量），高频操作缓存（降低成本）。

经验法则：80% 的简单问题用单体快速响应，20% 的复杂问题用模块化深度处理。

控制方式：Prompt-driven vs Code-driven（提示词驱动 vs 代码驱动）

Prompt-driven（提示词驱动）：让 LLM 自己决策

核心思想：通过精心设计的 Prompt，让 LLM 自主完成推理、规划、工具调用等决策。

想象给助手一本"工作手册"（Prompt），然后完全信任它按手册自主工作，不干预具体的执行流程。

```
system_prompt = """
你是一个智能助手，可以使用以下工具：
- search(query): 搜索网络
- calculator(expr): 计算数学表达式

遇到问题时，你可以：
1. 先思考需要什么信息
2. 选择合适的工具获取信息
3. 根据结果继续思考或给出答案

你可以自由决定何时使用工具、使用哪个工具。
"""

# LLM 根据 Prompt 自主决定整个流程
response = llm.chat([{"role": "system", "content": system_prompt}, ...])
```

优势在于灵活性极高（LLM 可以创造性地组合工具），适应性强（自动处理意外情况），开发快（只需写好 Prompt）。

劣势在于不稳定（LLM 可能不按套路出牌），难调试（出错了不知道是哪步思考有问题），成本高（每次决策都要调 LLM，调试时尤其浪费 Tokens）。

Code-driven（代码驱动）：代码控制流程

核心思想：用代码明确定义 Agent 的行为逻辑，LLM 只负责理解和生成。

```
def code_driven_agent(user_input):
    # 代码控制流程
    intent = classify_intent(user_input)  # LLM 分类

    if intent == "查天气":
        city = extract_city(user_input)  # LLM 提取
        weather = get_weather(city)  # 工具调用
        return format_weather_response(weather)  # LLM 生成回复

    elif intent == "订票":
        # 明确的流程步骤
        ...
```

优势在于稳定可控（流程完全由代码决定），易调试（每一步都清晰可见），性能好（只在必要时调用 LLM），成本较低。

劣势在于灵活性差（只能处理预定义的场景），开发重（每个场景都要写代码），扩展麻烦（新增场景要改代码）。

混合模式：最佳实践

现代 Agent 的主流做法：用代码定义主流程（保证稳定性），在关键节点让 LLM 自主决策（保留灵活性）。

示例（混合模式）：

示例（混合模式）

```
def hybrid_agent(user_input):
    # Code-driven: 明确的流程框架
    intent = classify_intent(user_input)

    if intent == "复杂查询":
        # Prompt-driven: 让 LLM 自主规划
        plan = llm.generate_plan(user_input)
        results = execute_plan(plan)  # LLM 决定步骤
        return summarize(results)

    elif intent == "简单问答":
        # Code-driven: 直接处理
        return simple_qa(user_input)
```

经验法则：流程清晰的部分用代码（如登录验证），需要灵活应变的部分用 Prompt（如客户投诉处理）。

下节预告

学完了这些实现范式，接下来我们不再纸上谈兵。第8节将带你从零实现一个智能旅行规划Agent——用户只需说出预算、时间和偏好，Agent就能自动规划完整的旅行方案。这个项目将完整展示我们前面学习的所有能力：四步决策流程、Chain of Thought推理、记忆系统，以及如何选择合适的实现范式。代码不多，但五脏俱全，看完你就知道如何把理论变成能跑的Agent。

■ 学点英语

中文	English	音标	说明
单体智能体	Monolithic Agent	/ˌmɒnəˈlɪθɪk ˈeɪdʒənt/	用一个LLM承担所有角色，没有功能分工的架构
模块化智能体	Modular Agent	/ˈmɒdjʊlər ˈeɪdʒənt/	将Agent拆分为多个专业模块各司其职的架构
提示词驱动	Prompt-driven	/prɒmpt ˈdrɪvn/	通过Prompt让LLM自主完成决策的控制方式
代码驱动	Code-driven	/koʊd ˈdrɪvn/	用代码明确定义Agent行为逻辑的控制方式
混合范式	Hybrid Paradigm	/ˈhaɪbrɪd ˈpærədaɪm/	代码控流程+LLM做决策的Agent实现方式
系统提示词	System Prompt	/ˈsɪstəm prɒmpt/	定义Agent角色和行为的顶层指令文本

图片

